Computing Platforms for Robotics

- How to compare SBCs and MCU boards
- When to choose Raspberry Pi, Jetson Nano, or Arduino

Contents

Fundamentals

- SBC vs MCU board
- robot task decomposition

Platform deep dive

- Raspberry Pi
- Jetson Nano
- Arduino

Integration

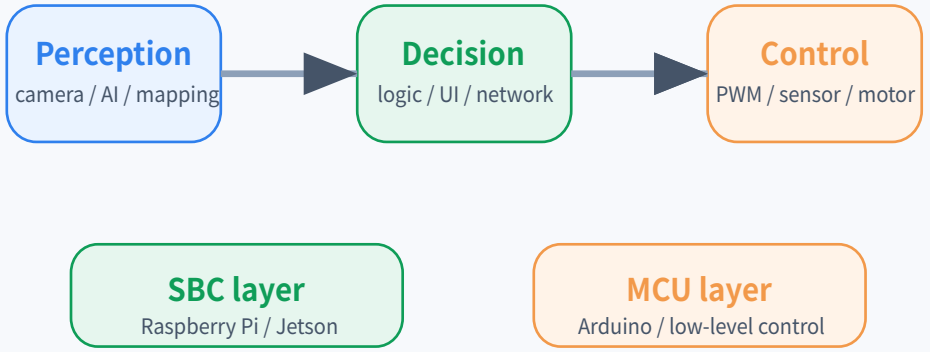
- hybrid architecture
- communication

Selection + Q&A

- decision guide
- case study

Why robots need more than one computing layer

- **Perception**: camera, mapping, vision, AI
- **Decision**: logic, planning, UI, networking
- **Control**: PWM, encoder, motor drivers, sensor timing



SBC vs microcontroller board

SBC

- Linux-capable board computer
- Good for files, UI, networking, camera, and higher-level software
- Examples: Raspberry Pi 4, Jetson Nano

Microcontroller board

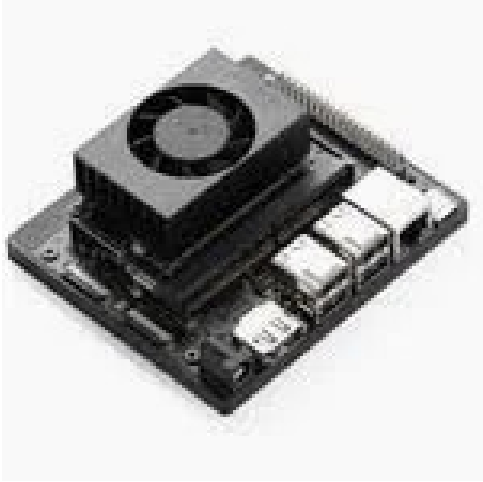
- Direct I/O and control loops
- Good for PWM, sensors, motor drivers, timing-sensitive tasks
- Example: Arduino Uno

Three familiar boards at a glance

Use the hardware family as an anchor before discussing strengths and weaknesses

Raspberry Pi 4

Linux , GPIO , camera



Jetson Nano

Vision , AI , GPU

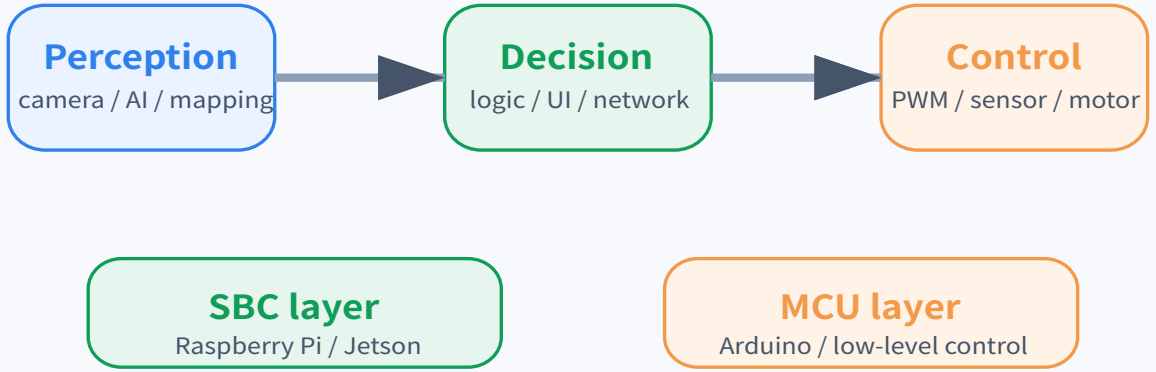
Arduino Uno

I/O , PWM , control

Students understand later details more easily once these three roles are visible side by side.

How robots split work across boards

Many real robots use high-level computing and low-level control together

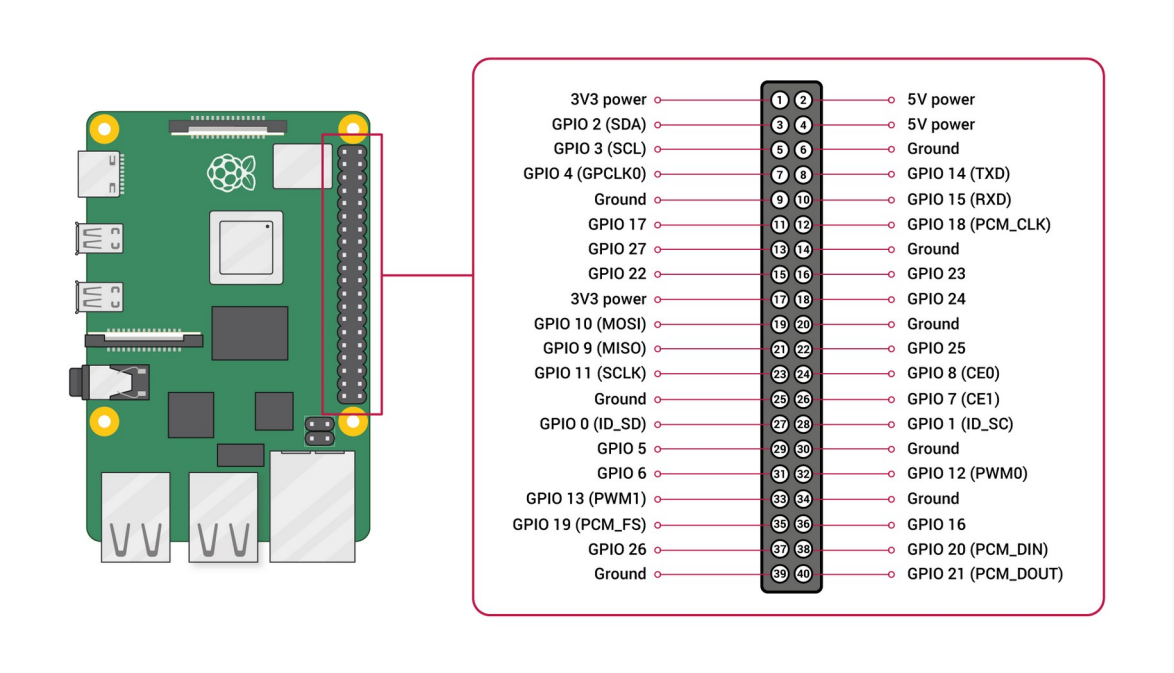


- SBC side: camera, AI, UI, logging, networking
- MCU side: motor control, encoder reading, sensor loops
- A hybrid design reduces overload on any single board

Raspberry Pi 4: the general robotics SBC

Raspberry Pi is often the easiest entry point for Linux-based robotics

- Broadcom BCM2711 quad-core Cortex-A72 SoC
- 1/2/4/8 GB LPDDR4 options
- 40-pin GPIO header
- CSI camera, DSI display, Wi-Fi, Bluetooth, Ethernet, USB 3.0



Raspberry Pi strengths and trade-offs

Strengths

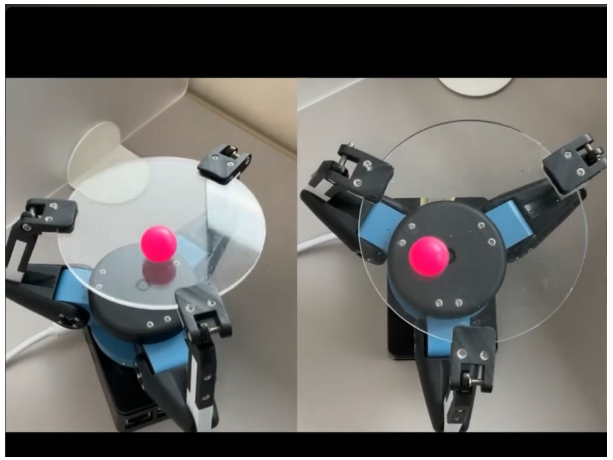
- Easy OS setup and Python workflow
- Strong accessory ecosystem
- Good for camera, web UI, logging, remote access

Trade-offs

- Less suitable for hard real-time loops
- OS setup adds complexity compared with Arduino
- Sometimes overkill for very simple robots

Good Raspberry Pi robot examples

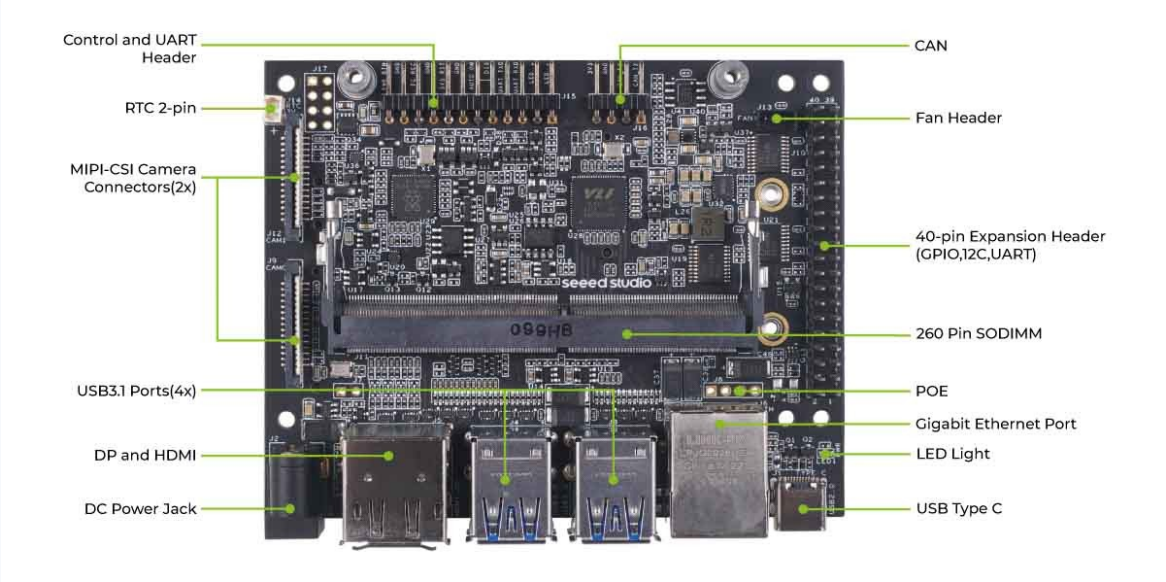
- Remote-controlled rover
- Camera streaming robot
- Sensor logging + dashboard
- Small research prototype



Jetson Nano: the AI / vision branch

Use Jetson when vision or inference is a defining requirement

- Quad-core ARM Cortex-A57 CPU
- 128 CUDA-core NVIDIA Maxwell GPU
- 4 GB LPDDR4 memory
- Small AI computer with a 40-pin expansion header



Jetson strengths and trade-offs

Jetson is strongest when GPU acceleration changes the project

Strengths

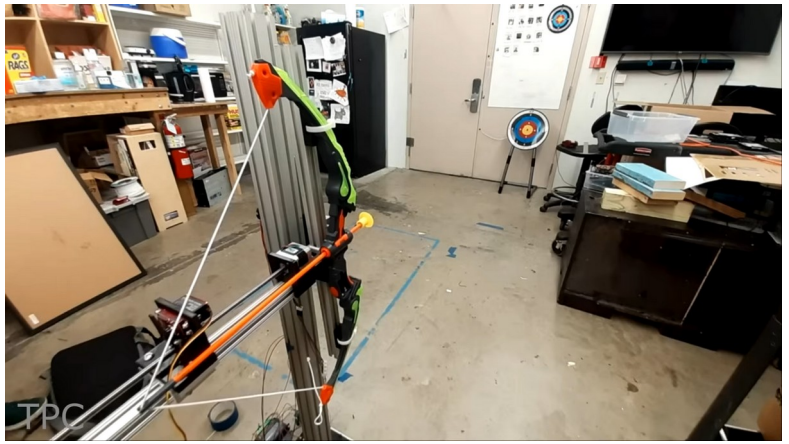
- Useful for object detection and vision pipelines
- Strong NVIDIA software ecosystem
- Good when AI is the reason the robot exists

Trade-offs

- More setup complexity
- Power and thermal planning matter more
- Overkill for simple sensing or motor tasks

Best-fit examples

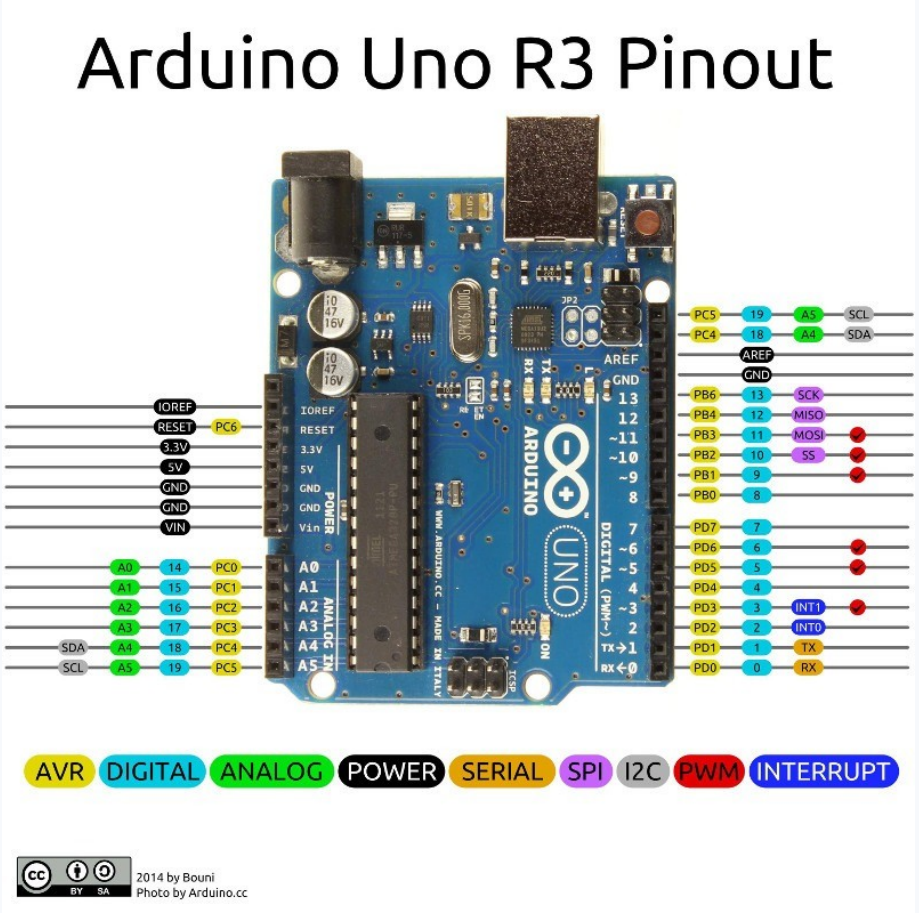
- Object-tracking robot
- Vision-based inspection system
- Edge AI demo platform



Arduino Uno: the clean control board example

Arduino remains one of the fastest ways to teach direct interaction with hardware

- ATmega328P-based microcontroller board
- 14 digital I/O pins
- 6 PWM outputs and 6 analog inputs
- Simple USB + direct I/O workflow



Arduino strengths and trade-offs

Arduino is powerful for simple control, but intentionally limited compared with SBCs

Strengths

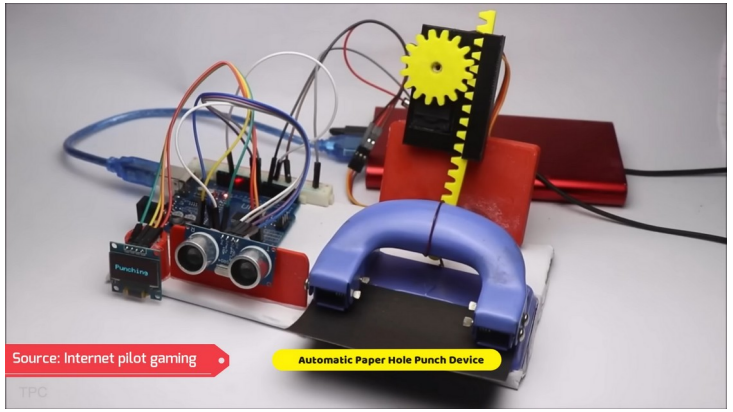
- Direct digital and analog I/O
- Simple PWM and motor/sensor labs
- Very low setup burden

Trade-offs

- No full Linux environment
- Limited memory and computing power
- Not the right board for camera or AI-heavy tasks

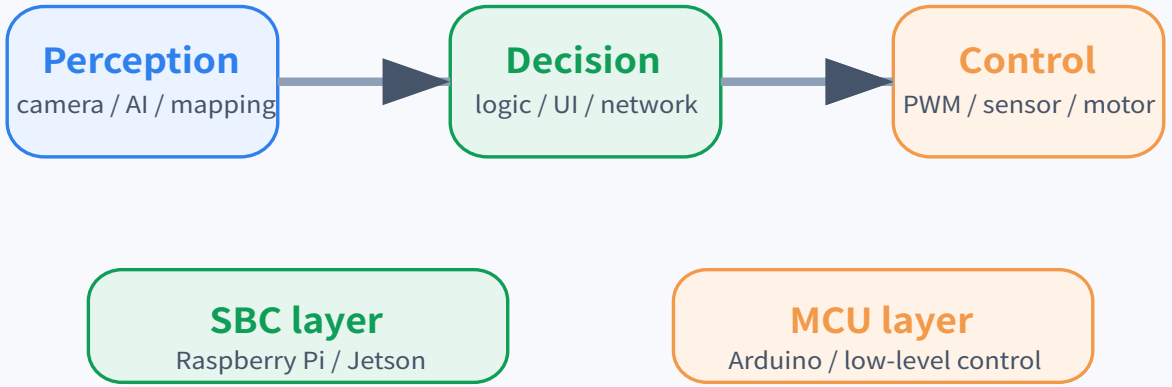
Typical Arduino robot jobs

- Motor driver control
- Sensor reading and filtering
- Encoder counting
- Servo actuation

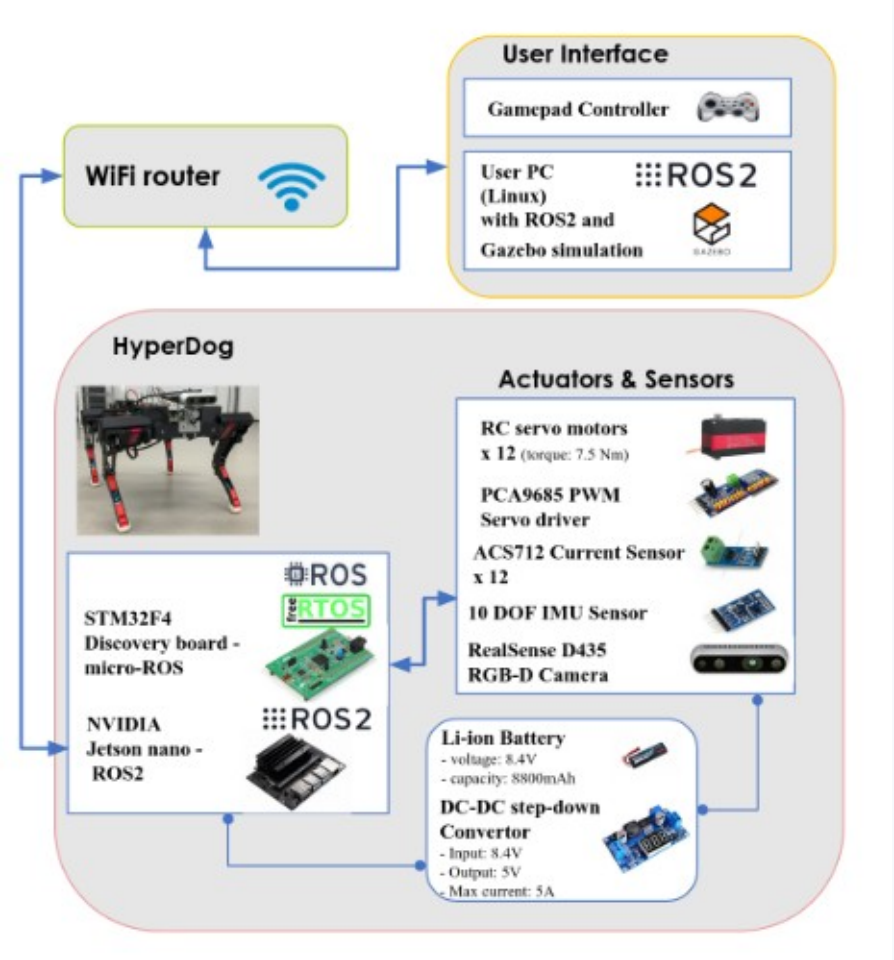


A common hybrid robot architecture

This is often the most realistic answer in practice



- SBC handles camera, UI, logging, networking, and high-level logic
- Arduino handles timing-sensitive I/O and actuator control
- The split makes both teaching and debugging easier



* HyperDog: An Open-Source Quadruped Robot Platform Based on ROS2 and micro-ROS

Wrap-up

Summary

Use the right platform for the right robotic job

Q&A

Raspberry Pi	general robotics SBC for Linux, GPIO, camera, and networking
Jetson Nano	AI / vision SBC when GPU acceleration matters
Arduino Uno	control board for direct I/O, PWM, and simple loops
Real robots	often combine an SBC with a microcontroller board

Pin-map of raspberry pi

01	3.3V DC Power	5V DC Power	02
03	GPIO02 (SDA1,I²C)	5V DC Power	04
05	GPIO03 (SDL1,I²C)	Ground	06
07	GPIO04 (GPCLK0)	GPIO14 (TXD0, UART)	08
09	Ground	GPIO15 (RXD0, UART)	10
11	GPIO17	GPIO18(PWM0)	12
13	GPIO27	Ground	14
15	GPIO22	GPIO23	16
17	3.3V DC Power	GPIO24	18
19	GPIO10 (SP10_MOSI)	Ground	20
21	GPIO09 (SP10_MISO)	GPIO25	22
23	GPIO11 (SP10_CLK)	GPIO08 (SPI0_CE0_N)	24
25	Ground	GPIO07 (SPI0_CE1_N)	26
27	GPIO00 (SDA0, I²C)	GPIO01 (SCL0, I²C)	28
29	GPIO05	Ground	30
31	GPIO06	GPIO12 (PWM0)	32
33	GPIO13 (PWM1)	Ground	34
35	GPIO19	GPIO16	36
37	GPIO26	GPIO20	38
39	Ground	GPIO21	40

Power + GND: supply sensors or small modules

Digital input: read buttons, switches, encoder signals, and digital sensors

Digital output: control LEDs, relays, and motor-driver input pins

I²C / SPI / UART: communicate with displays, IMUs, ADCs, and microcontrollers

PWM: generate pulse-width signals for speed control, dimming, or servo-style control

*** Important notes

Raspberry Pi GPIO logic is 3.3V

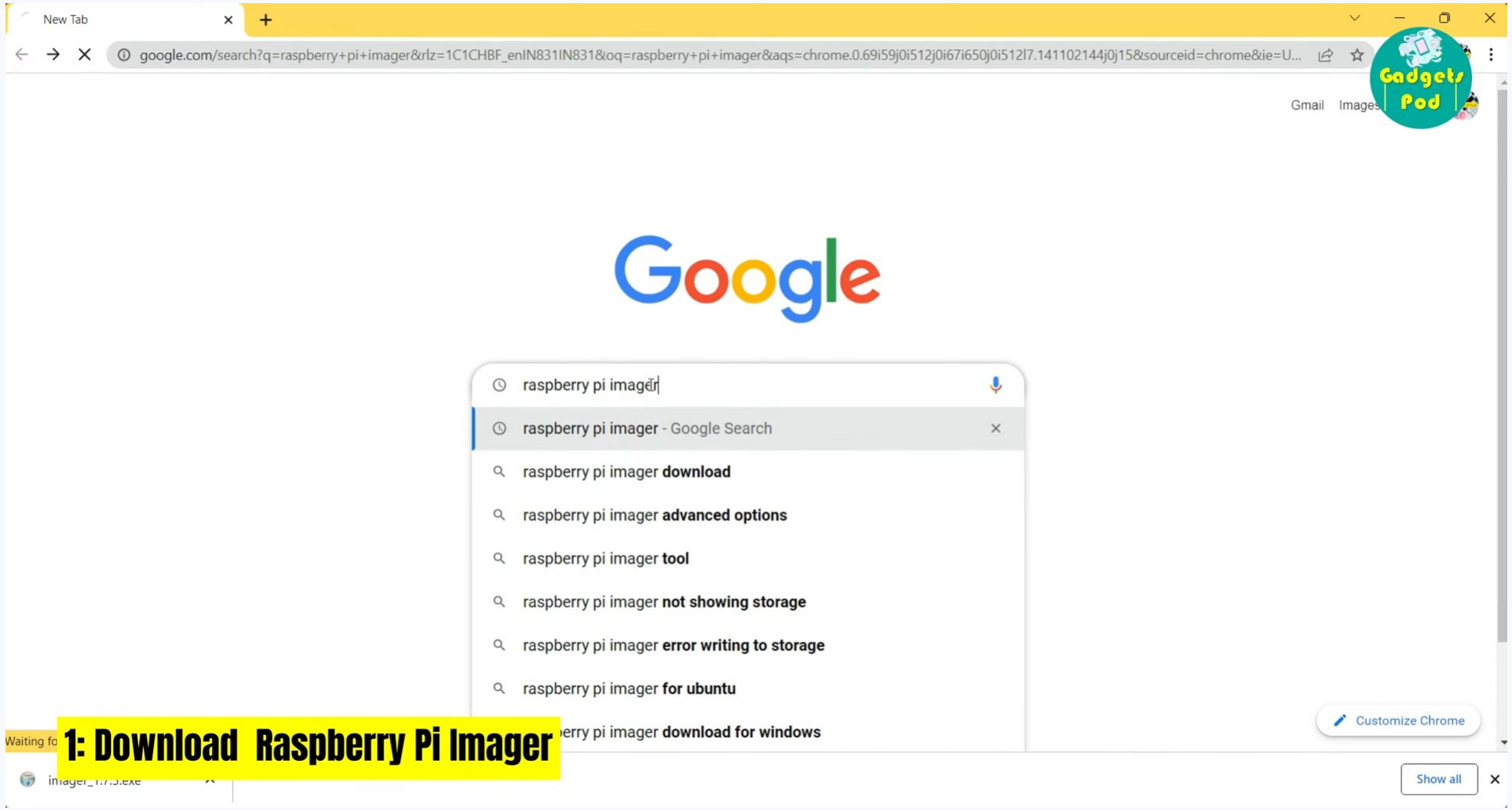
Do not connect 5V signals directly to GPIO pins

Do not drive motors directly from GPIO; use a motor driver or interface board instead

Physical pin numbers and BCM GPIO numbers are different, so always check the pin map before wiring

Install os on raspberry pi

1. Download Raspberry Pi imager on your labtop

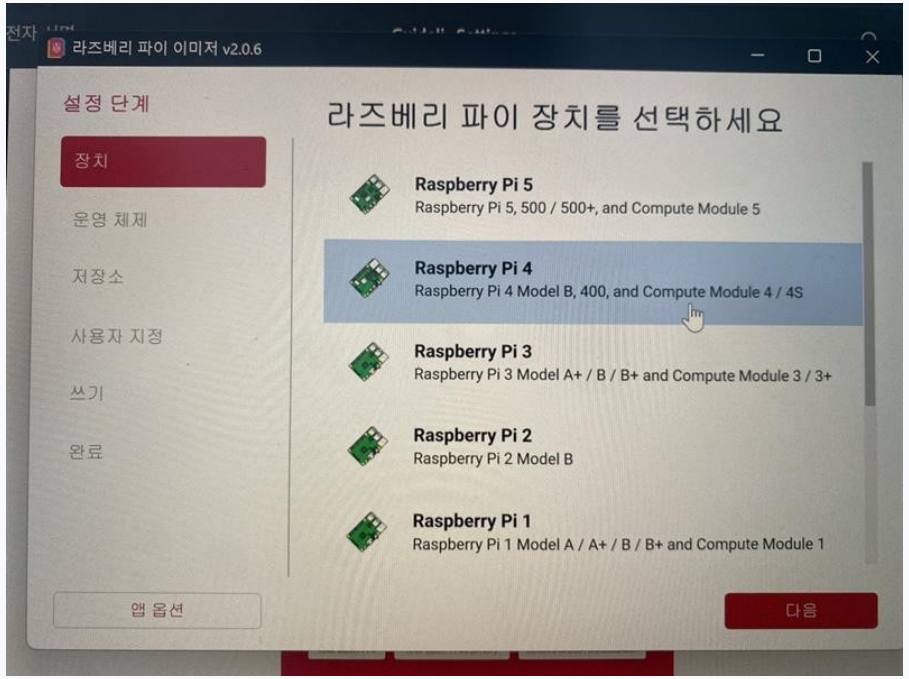


Install os on raspberry pi

2. Connect sd card on your labtop with adapter

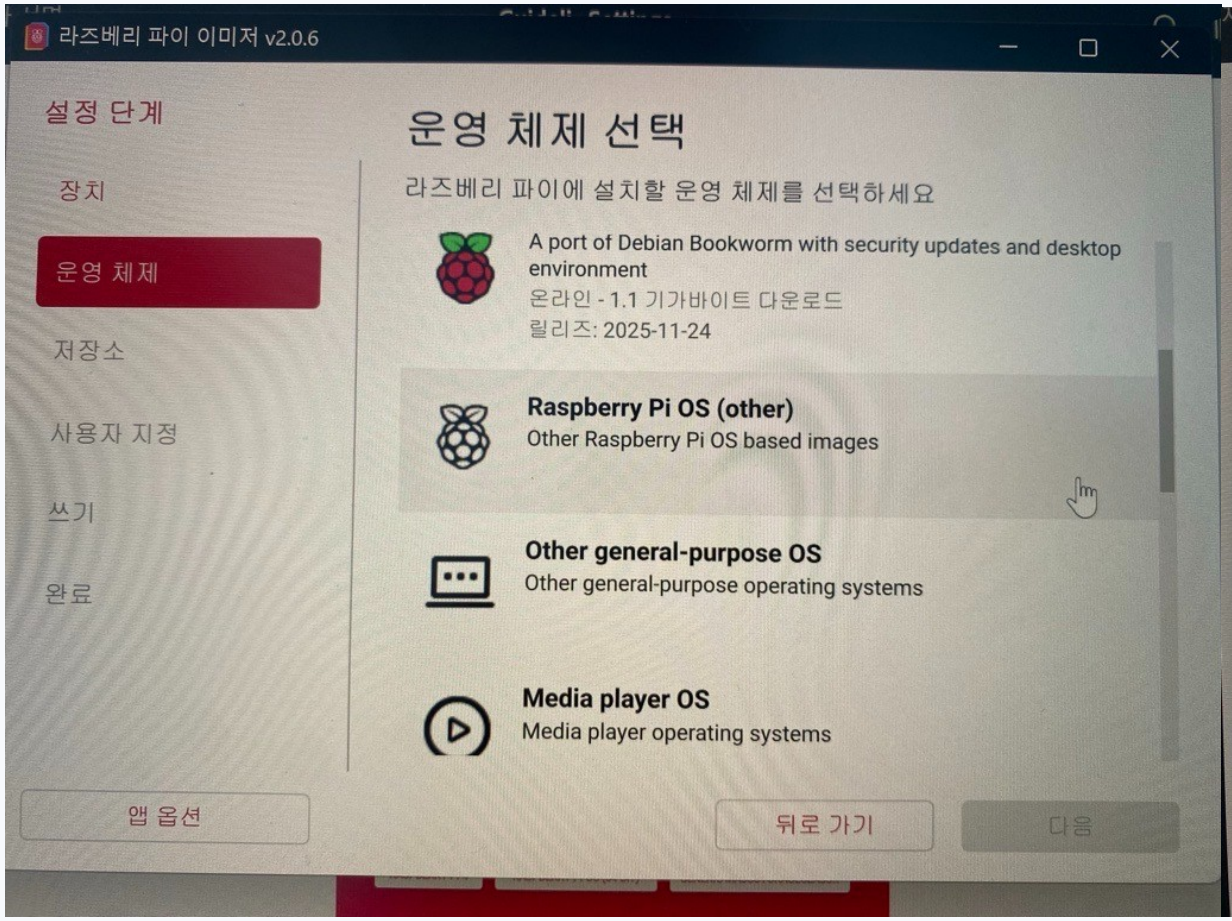
Install os on raspberry pi

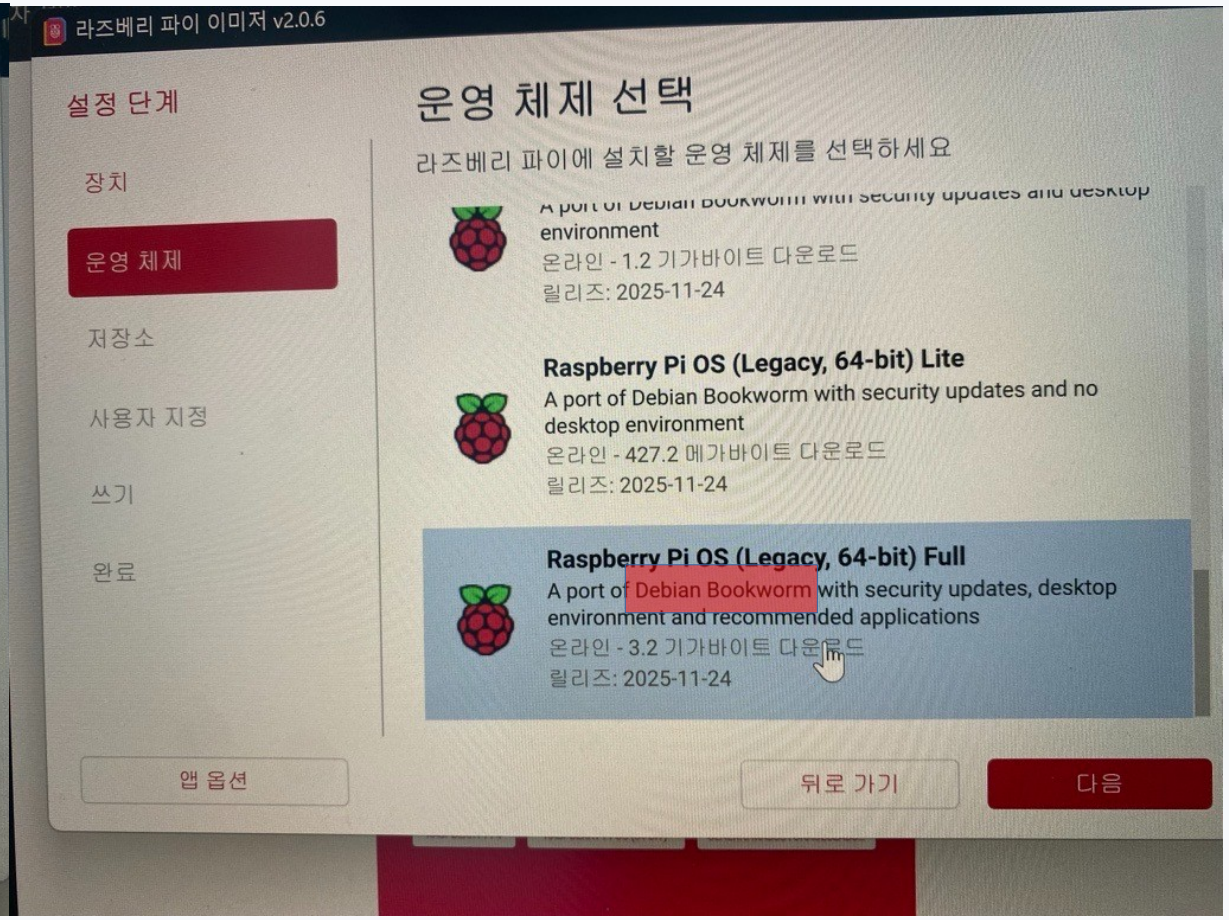
3. Run raspberry pi imager and choose os



Install os on raspberry pi

3. Run raspberry pi imager and choose os (Raspberry pi os is highly recommended!! – make sure that Debian bookwarm is selected)

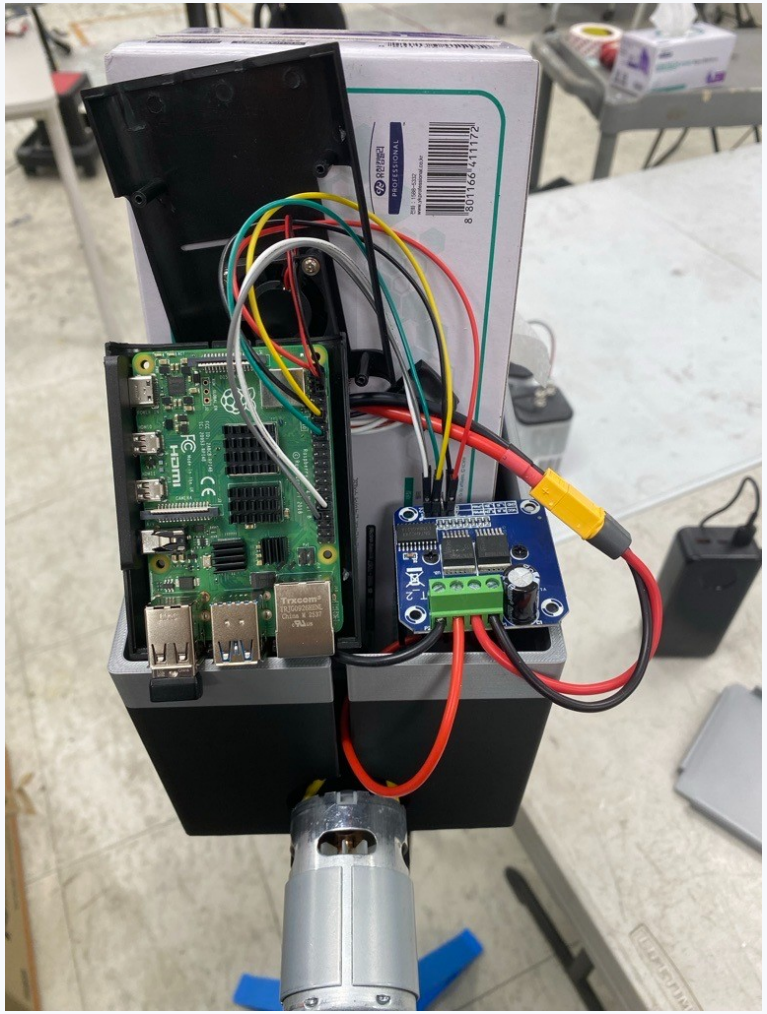




Raspberry Pi OS is still Linux, but it is the Linux distribution most closely optimized and supported for Raspberry Pi hardware, so it is usually the easiest choice for beginners

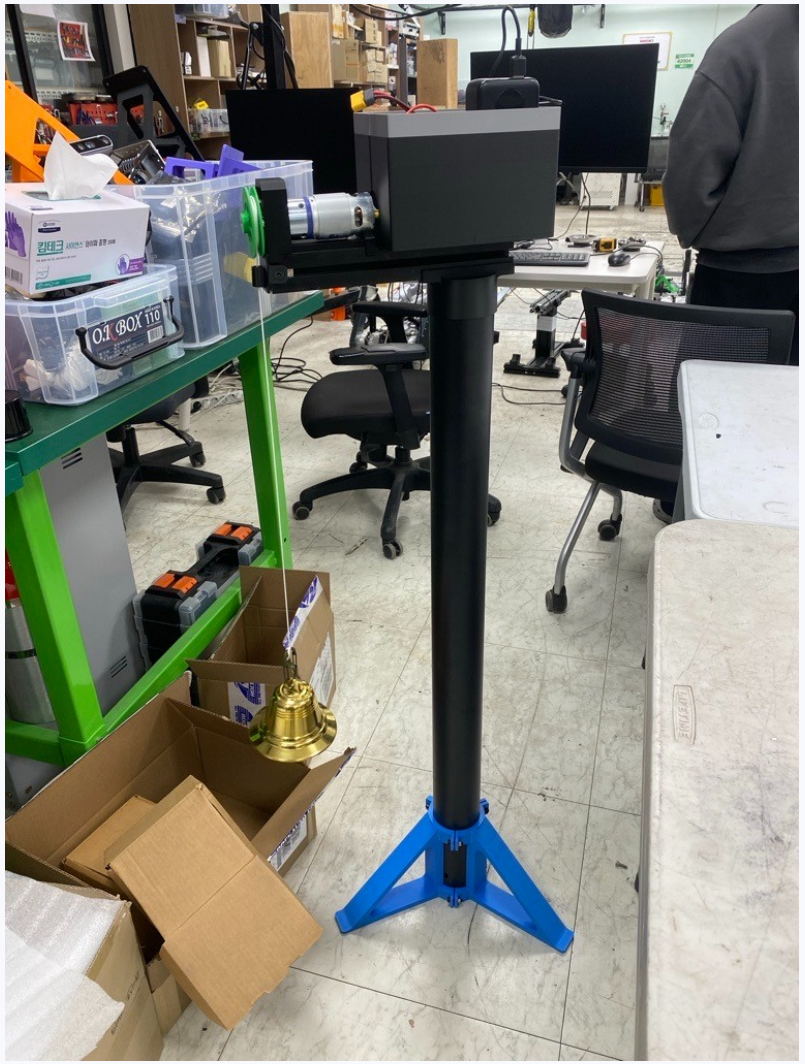
Motor control example

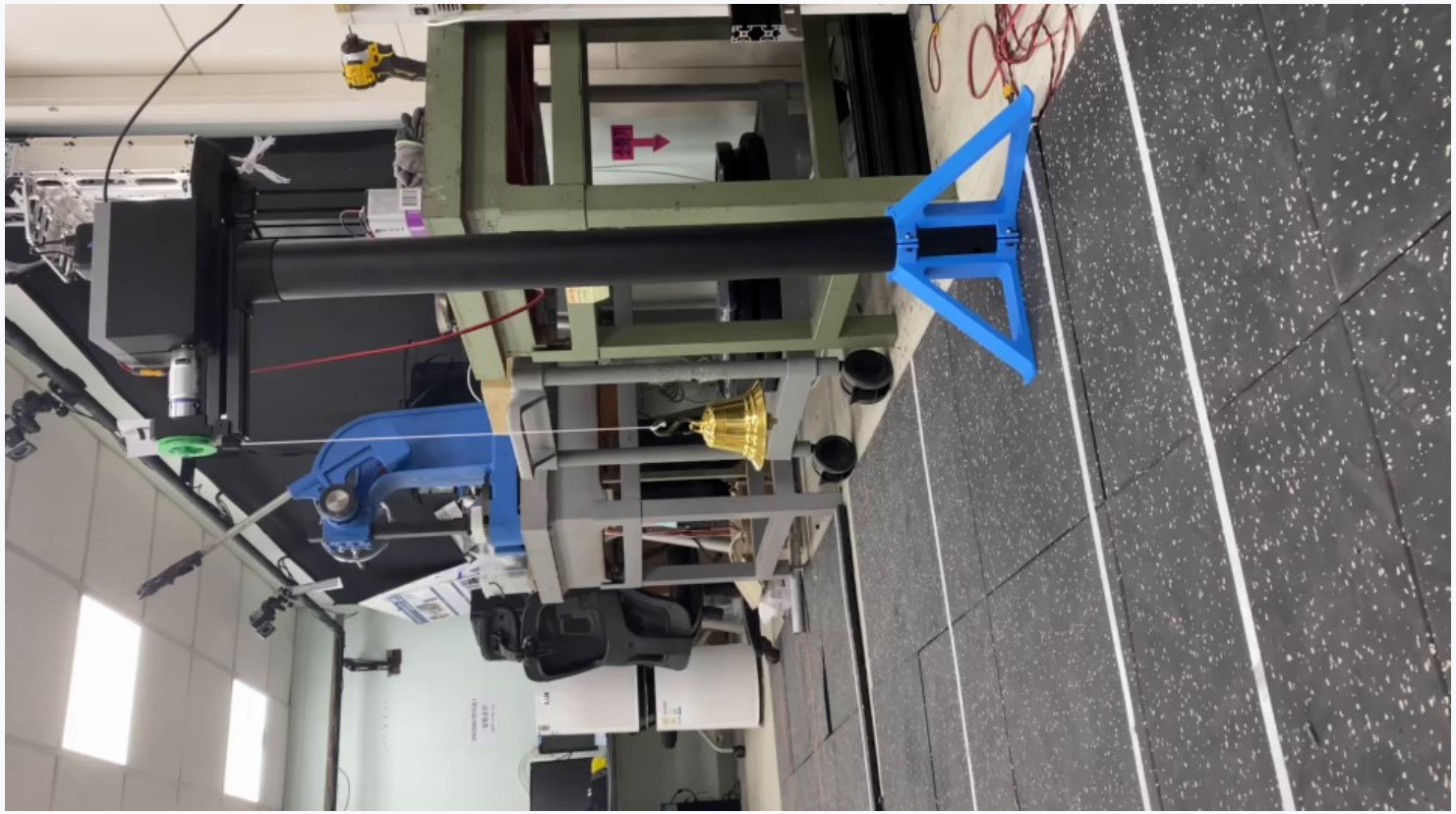
- Bell Reciprocating Mechanism for the Final Demo



Motor control example

- Bell Reciprocating Mechanism for the Final Demo





Motor control example

- Bell Reciprocating Mechanism for the Final Demo

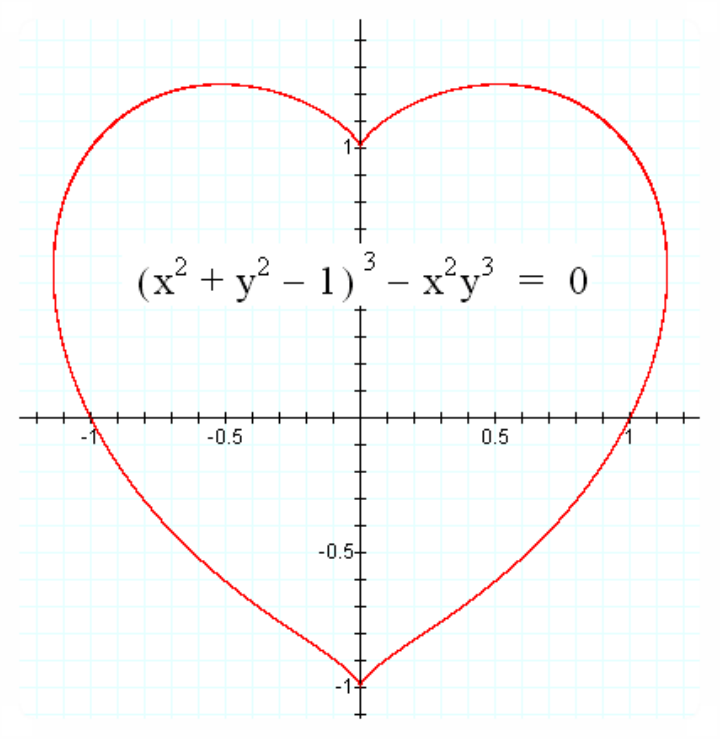
What might be the disadvantages of this design?

Motor control example

- Settings to Make Your Controller More Exciting

1. Predict function of motion

-A framework in which the bell's amplitude function is predefined, the bell motion is observed during the initial few seconds using the YOLO algorithm to estimate the coefficients, and these estimated coefficients are incorporated into the control algorithm in real time.



Motor control example

- Settings to Make Your Controller More Exciting

2. Projectile-Avoidance Bell Controller

Motor control example

- Settings to Make Your Controller More Exciting

Any recommendations?

Q & A